Assignment 1 — Data Preprocessing Report

E-commerce Customer Dataset: Cleaning & Preparation for Churn Prediction

1. Business Scenario & Dataset

An e-commerce company wants to predict whether a customer will make another purchase (`ChurnNextMonth`). Before any model can be built, the raw customer dataset must be cleaned and prepared. The raw file contains **2,200 customer records across 25 attributes**, mixing demographics (`Age`, `Gender`, `City`, `Country`), financial/behavioural fields (`AnnualIncome_INR`, `SpendingScore_1_100`, `AvgBasketValue_INR`, purchase counts, `LoyaltyPoints`), engagement fields (`MembershipTier`, `VisitFrequency`, `CouponUsed`, `SatisfactionRating_1_5`), channel metadata (`DeviceType`, `EmailProvider`, `PaymentMethod`), two date fields, two identifiers, and the target label.

Of the 25 attributes, **9 are numerical** (`Age`, `AnnualIncome_INR`, `SpendingScore_1_100`, `AvgBasketValue_INR`, `TotalPurchases`, `OnlinePurchases`, `StorePurchases`, `LoyaltyPoints`, `SatisfactionRating_1_5`), **11 are categorical** (`Gender`, `IncomeCurrency`, `MembershipTier`, `VisitFrequency`, `PreferredCategory`, `City`, `Country`, `EmailProvider`, `DeviceType`, `PaymentMethod`, `CouponUsed`), 2 are dates, 2 are identifiers, and 1 is the binary target (`ChurnNextMonth`).

2. Data Quality Assessment

A systematic scan of the raw data surfaced problems in almost every category the assignment asks about:

Problem	Column(s)	Count	Impact
Missing values	18 of 25 columns	worst: `Country` 362 (16.5%), `IncomeCurrency` 318 (14.5%), `CouponUsed` 265 (12.0%)	Reduces usable data; biases summary statistics
Fully duplicated rows	entire record	64	Double-counts the same customer
Duplicate primary key	`CustomerID`	113 repeated IDs, **47 with genuinely conflicting data**	Breaks the one-row-per-customer assumption
Inconsistent categories	`Gender`, `MembershipTier`, `City`, `PaymentMethod`, `CouponUsed`, etc.	6-30 raw spellings per column	Fragments categories, corrupts group-bys and one-hot encoding
Invalid / impossible values	`Age` (35 rows <0 or >100), `AnnualIncome_INR` (5 <0), `AvgBasketValue_INR` (24 <0), `TotalPurchases` (18 <0), `LoyaltyPoints` (5 <0), `SatisfactionRating_1_5` (49 outside 1-5)	~136 cells	Physically impossible, distorts statistics/models
Logical inconsistency	`TotalPurchases` vs `Online + Store`	**1,062 rows (48.3%)**	Nearly half the data disagrees with itself
Extreme outliers (IQR rule)	`AnnualIncome_INR` (87), `AvgBasketValue_INR` (139), `TotalPurchases` (111), `LoyaltyPoints` (19)	up to 139 rows	Long tails dominate scale-sensitive / distance-based models
Data-type issues	`JoinDate`, `LastPurchaseDate` stored as text in 4 mixed formats; `CouponUsed` mixes numeric/text booleans	all rows	Blocks tenure/recency calculations and boolean filtering

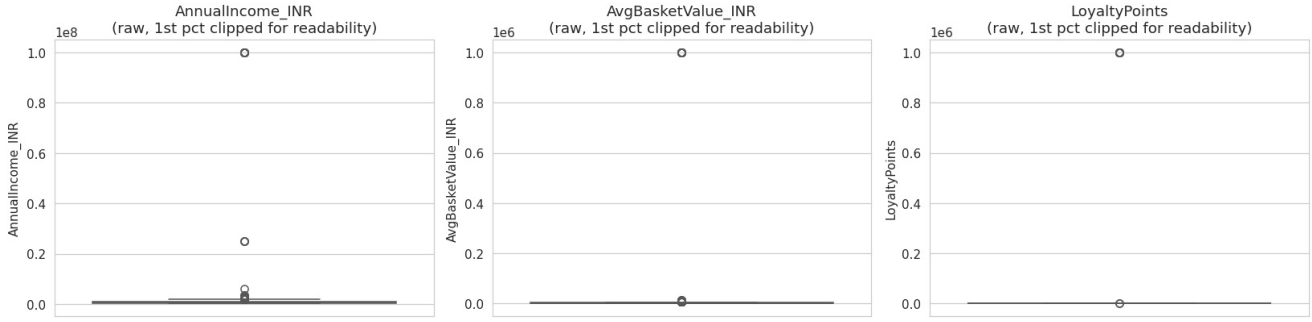
3. Data Cleaning

Cleaning was carried out as a sequence of 18 explicit, auditable steps (each with a before/after check and rationale in the notebook). The key moves:

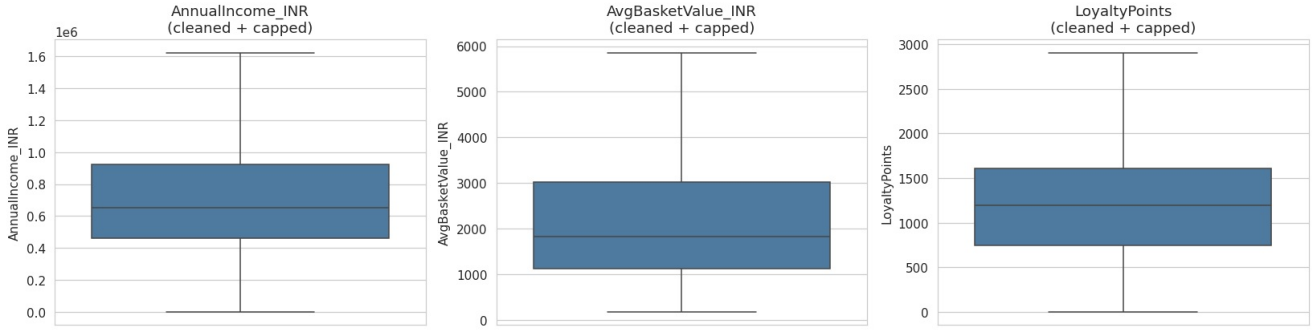
1. **Duplicates.** Dropped 64 exact duplicate rows. Regenerated a guaranteed-unique `CustomerID` for every row (preserving the raw value in `CustomerID_Original`), since 47 of the 113 repeated IDs belonged to genuinely different customers.
2. **Category standardization.** Stripped stray whitespace and unified case/typo variants in `Gender` (→ Male/Female/Other), `MembershipTier` (fixed `"Silvr"` → Silver), `VisitFrequency`, `PreferredCategory`, `City` (`"Mumbay"`→Mumbai, `"Bangalore"`→Bengaluru, `"Calcutta"`→Kolkata, `"Hyd"`→Hyderabad), `Country` (`"Bharat"`/`"IN"`→India), `IncomeCurrency` (`"Rs"`/`"₹"`/`"USD"`→INR, since the mislabeled “USD” rows are on the same magnitude as the INR rows), `DeviceType`, `PaymentMethod`, and `CouponUsed` (unified to a clean 0/1 flag).

3. **Dates.** Parsed both date columns (4 mixed string formats) into proper `datetime64` values. The few unparseable `JoinDate` values were imputed with the median join date; `LastPurchaseDate` gaps were **deliberately left blank**, since a customer may genuinely have no recorded purchase yet — fabricating a date would invent behavioural history.
4. **Invalid values → missing → impute.** Impossible values (negative age/income/basket value/loyalty points, out-of-range ratings) were converted to `NaN`, then imputed with the column median (numeric) or mode (categorical). `TotalPurchases` was **rebuilt** as `OnlinePurchases + StorePurchases` rather than patched, resolving the 1,062-row inconsistency at its root.
5. **Target rows.** The 27 rows with a missing `ChurnNextMonth` label were dropped — imputing a label would fabricate ground truth for a supervised model.
6. **Outlier treatment.** Residual extreme-but-possible values in `AnnualIncome_INR`, `AvgBasketValue_INR`, and `LoyaltyPoints` were winsorized (capped) at 1.5×IQR, bounding their influence without discarding rows.

Result: the cleaned dataset has **2,109 rows × 26 columns** (91 rows removed: 64 exact duplicates + 27 missing-label rows), with only the intentionally-preserved `LastPurchaseDate` gaps (73 rows) remaining as missing.



Outlier boxplots before cleaning (extreme values dominate the scale)

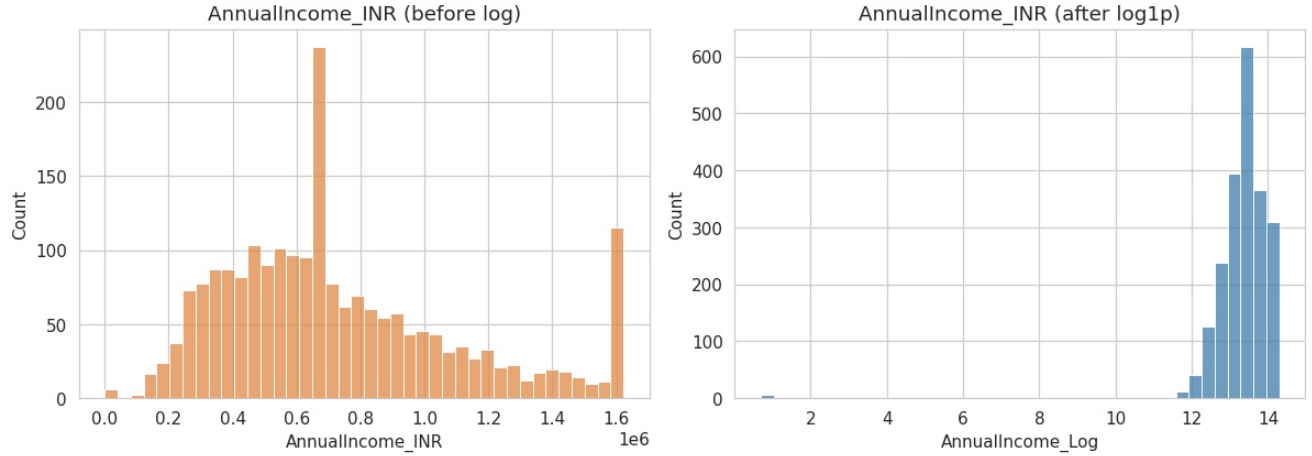


Outlier boxplots after cleaning and 1.5×IQR capping

4. Data Transformation

Five transformations were applied and compared before/after:

- **Min-Max normalization** on `SpendingScore_1_100` → rescaled to [0, 1].
- **Standardization (z-score)** on `AnnualIncome_INR` and `AvgBasketValue_INR` → mean 0, std 1, putting very different-scaled columns (lakhs vs. hundreds of rupees) on equal footing.
- **Robust scaling** on `LoyaltyPoints`, using the median/IQR instead of mean/std since some skew remains even after capping.
- **Log transformation** (`log1p`) on `AnnualIncome_INR`, which reduced its skewness from a strongly right-skewed distribution to a much more symmetric one (see figure below).
- **Encoding:** `MembershipTier` was **label-encoded** (Basic=0 < Silver=1 < Gold=2 < Platinum=3) since it has a natural order; `Gender`, `PaymentMethod`, and `DeviceType` were **one-hot encoded** since they have no natural order.



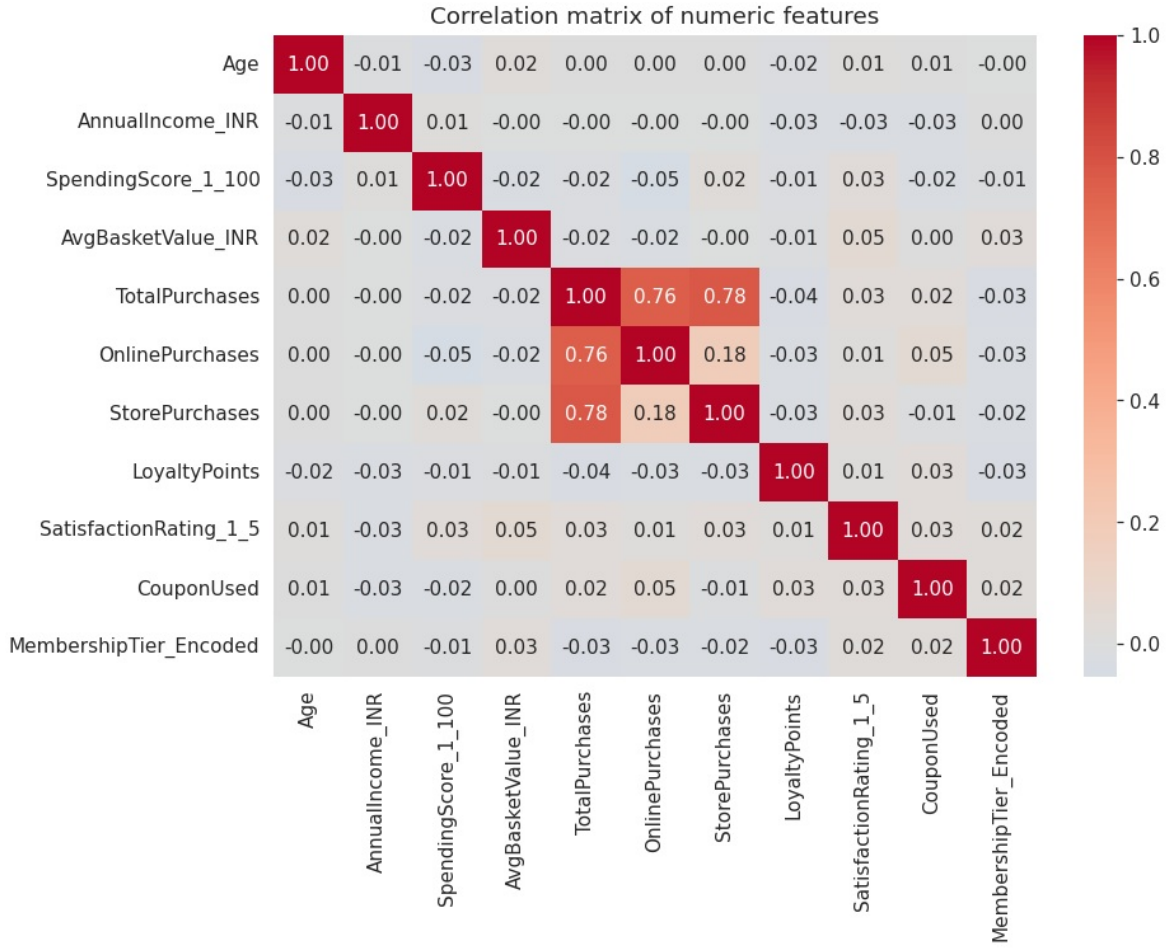
AnnualIncome_INR distribution before vs. after log1p transformation

5. Data Reduction

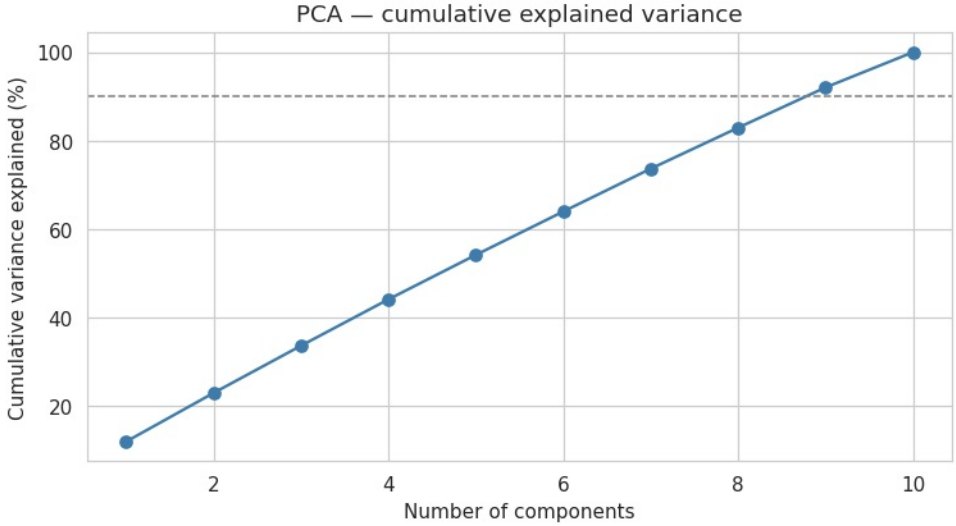
Two reduction techniques were applied to the 10 candidate numeric features:

(A) Correlation-based / structural feature removal. No pair crossed the 0.85 Pearson correlation threshold on its own (the strongest pairwise correlations were `TotalPurchases–OnlinePurchases` at 0.76 and `TotalPurchases–StorePurchases` at 0.78). However, `TotalPurchases` is *structurally* redundant — by construction it is the exact sum of the other two — so it was removed anyway. This is a useful lesson: a purely statistical correlation cutoff can miss redundancy that domain knowledge about how a feature was derived would catch immediately.

(B) Principal Component Analysis. On the remaining 10 features (standardized), **9 principal components were needed to retain 90%+ of the variance (92.1% at 9 components)**. This near-linear growth in explained variance indicates the remaining features are largely independent of one another — expected, since demographics, spending behaviour, loyalty, and satisfaction are conceptually distinct dimensions of a customer. PCA is offered as an option for algorithms sensitive to dimensionality or multicollinearity, even though the dataset does not compress dramatically.



Correlation matrix of the numeric feature set used for reduction

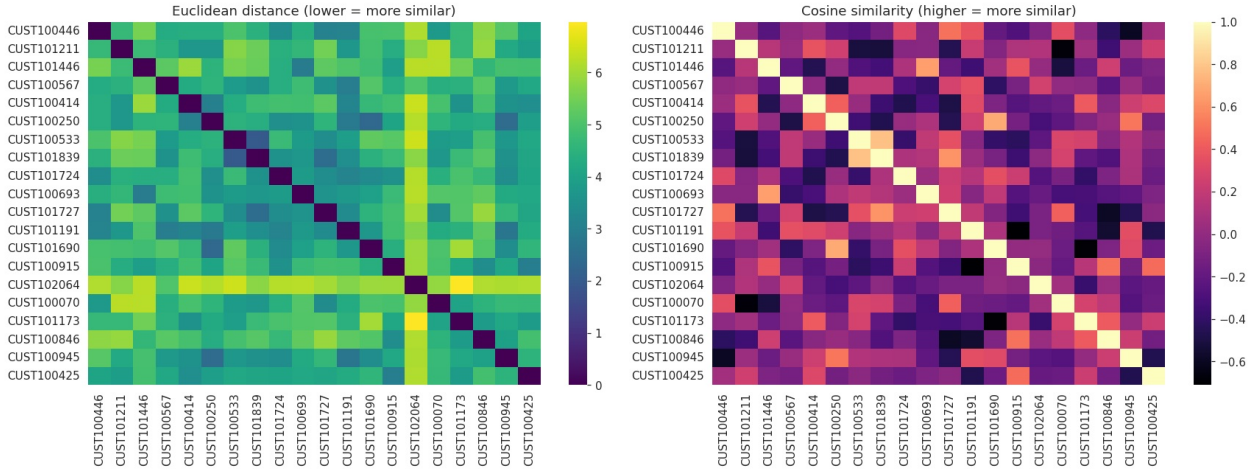


PCA cumulative explained variance

Size comparison: 10 numeric features → 9 after correlation/structural removal → 9 principal components (92.1% variance) if PCA is applied for a distance-sensitive model.

6. Proximity Measures

20 customers were sampled and their standardized numeric profiles compared using **Euclidean distance** and **Cosine similarity**.



Euclidean distance and Cosine similarity heatmaps for 20 sampled customers

Both measures agreed on the same most-similar pair: **CUST100533 and CUST101839** (Euclidean distance = 1.95, Cosine similarity = 0.78). These two customers have closely matching income (₹9.06L vs ₹9.84L), similar age (33 vs 40), similar spending score (53 vs 56), and identical membership tier and coupon usage, illustrating that Euclidean distance picks up customers who are close on the *absolute* scale of every attribute simultaneously. Cosine similarity instead rewards customers whose attributes move together in the same *relative pattern*, even at different absolute levels — useful when the goal is to find customers with a similar behavioural "shape" (e.g. for look-alike marketing) rather than customers who are simply numerically close.

7. Reflection

Major challenges. The biggest challenge was the sheer variety of quality problems packed into a fairly small (2,200-row) table: nearly every categorical column had its own inconsistent-casing/whitespace/typo pattern (e.g. 'Silvr', ' GOLD ', 'Mumbay', 'Bharat'), several numeric columns contained physically impossible values (negative ages, ages above 100, negative income, ratings outside the stated 1-5 scale), and roughly half the rows had a TotalPurchases value that contradicted its own component counts (OnlinePurchases + StorePurchases). On top of that, the two date columns mixed at least four different string formats, and 113 CustomerID values were reused across genuinely different customers — a primary-key integrity problem that is easy to overlook if you assume an ID column is trustworthy by default. Untangling which anomalies were genuine data noise (safe to fix or impute) versus which were legitimate rare events (safe to keep, only scale/cap) required going column-by-column rather than applying one blanket rule.

Highest-impact step. Reconciling TotalPurchases with OnlinePurchases + StorePurchases was arguably the single highest-impact step: it touched ~48% of rows and replaced an unreliable, occasionally absurd value (up to 9,999 purchases) with a value that is guaranteed to be internally consistent and directly interpretable. The second most effective was standardizing the categorical columns — collapsing Gender, MembershipTier,City, PaymentMethod, etc. down to their true number of categories dramatically improves the quality of any group-by analysis, and is essential before one-hot encoding (otherwise the model would learn spurious "different" categories for 'Gold' and ' GOLD ').

Remaining problems. A handful of LastPurchaseDate values are still missing — we deliberately chose not to fabricate a purchase date, so any recency-based feature will need to explicitly handle these as "unknown," not as "long ago." The dataset also still contains some genuinely extreme (but not impossible) values after winsorization at the 1.5xIQR bound; this reduces their influence but does not remove the underlying uncertainty about whether they represent real high-value customers or further data-entry noise. Finally, standardizing CustomerID collisions by regenerating a surrogate key resolves the symmetry problem but cannot recover which of the conflicting original records is "correct".

Recommended algorithm. For predicting ChurnNextMonth, a gradient-boosted tree model (e.g. XGBoost / LightGBM) is a strong first choice: tree ensembles handle the mix of numeric and categorical features gracefully, are naturally robust to the residual outliers and skew that remain even after cleaning, and typically give strong out-of-the-box performance on tabular churn-style problems without requiring extensive further scaling. A regularized logistic regression is a good complementary baseline for interpretability since coefficients are directly explainable, especially once the scaled/encoded features are used.

Before deployment. Before shipping a production model we would want to:
(1) re-validate the cleaning rules against a live, continuously-refreshed data feed, since new unseen category spellings or date formats will keep appearing;
(2) engineer explicit tenure (today - JoinDate) and recency (today - LastPurchaseDate) features now that the dates are true datetimes;
(3) monitor for target/label leakage — e.g. confirm that ChurnNextMonth genuinely reflects future behaviour and was not partly derived from features already in the table;
(4) set up automated data-quality checks (schema validation, range checks, referential-integrity checks on CustomerID) so the issues found manually are caught automatically on every new data load;
(5) periodically re-fit the scalers/encoders (or persist them) so that new data is transformed exactly the same way the training data was.